DSA STUDY BLUEPRINT SERIES

36. Recursion Part2 Fibonacci Binary Search

Fibonacci Calculations and Recursive Binary Search

Section 05 • C++ Core Data Structures & Algorithms

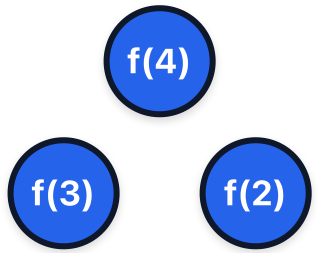
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Recurrence:** Defining function outputs based on smaller inputs.
- **Tree:** Recursion splits form tree structures (e.g. Fibonacci: 2 branches).

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
int recBS(int arr[], int s, int e, int target) {
    if (s > e) return -1;
    int mid = s + (e - s) / 2;
    if (arr[mid] == target) return mid;
    if (arr[mid] < target) return recBS(arr, mid + 1, e, target);
    return recBS(arr, s, mid - 1, target);
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Recursive BS	O(log N) time, O(log N) stack
Fibonacci Recursive	$O(2^N)$ time

Key Practices & Gotchas

- **Important:** Use memoization to optimize recursive Fibonacci to O(N) time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Binary Search looking for target `47` in `[12, 24, 35, 47, 58, 69, 80]`:

Iteration	Search Range Indices	Mid Index Calculation	Element at Mid	Comparison & Action
1	[0, 6]	mid = 0 + (6-0)/2 = 3	arr[3] = 47	47 == 47 (Target found at index 3!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Lower/Upper Bounds:** Binary search variations resolve the first element not-less-than target (lower bound) or strictly-greater-than target (upper bound).
- **Search Space Boundary:** Ensure checking `start ≤ end` conditions and compute mid indices as `s + (e-s)/2` to prevent integer overflow.
- **Duplicates:** Adjust left/right boundaries inwards to trace duplicate values edges.
- **Related LeetCode Problems:** #34 Find First and Last Position of Element in Sorted Array, #704 Binary Search, #33 Search in Rotated Sorted Array.